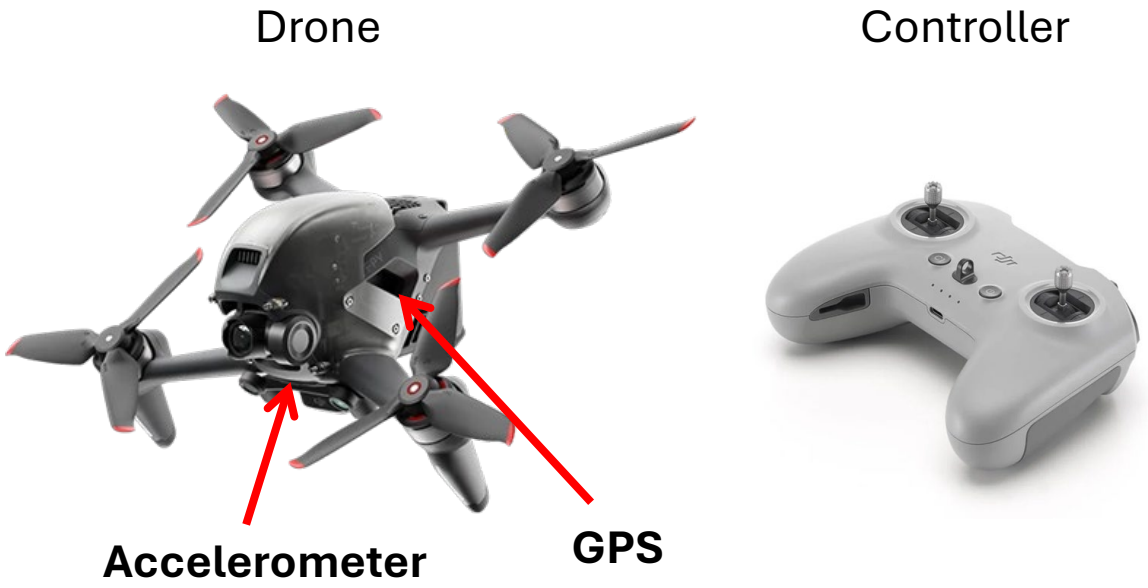
DSAI4201
Selected Topics in Data Science

Reinforcement Learning

What is Reinforcement Learning

Example: Autonomous Drone

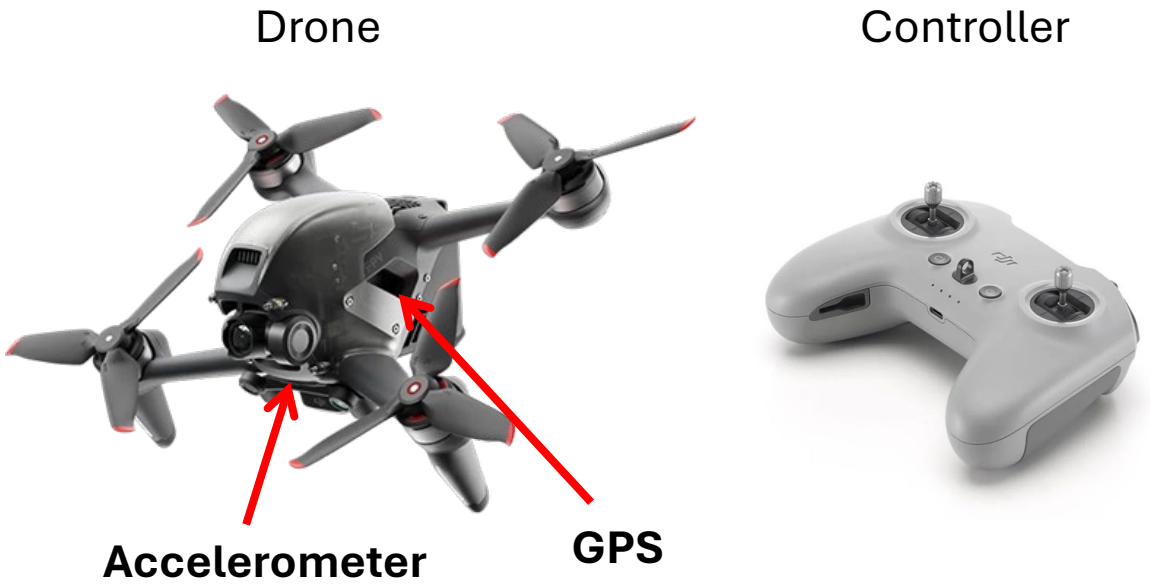


- The drone senses its **state** (e.g., position and motion) 10 times per second.
- **How to fly it** automatically?

What is Reinforcement Learning

Example: Autonomous Drone

- Model chooses how to move the control sticks (selects an action) based on current state.
- The drone tries the chosen action and sees what happens. It gets:
  - a **reward** for staying balanced and stable.
  - a **penalty** for tilting (losing balance) or crashing.
- Over many trials, it learns which actions keep it flying.
- Eventually, it can hover and fly on its own.



- The drone senses its **state** (e.g., position and motion) 10 times per second.
- **How to fly it** automatically?

What is Reinforcement Learning

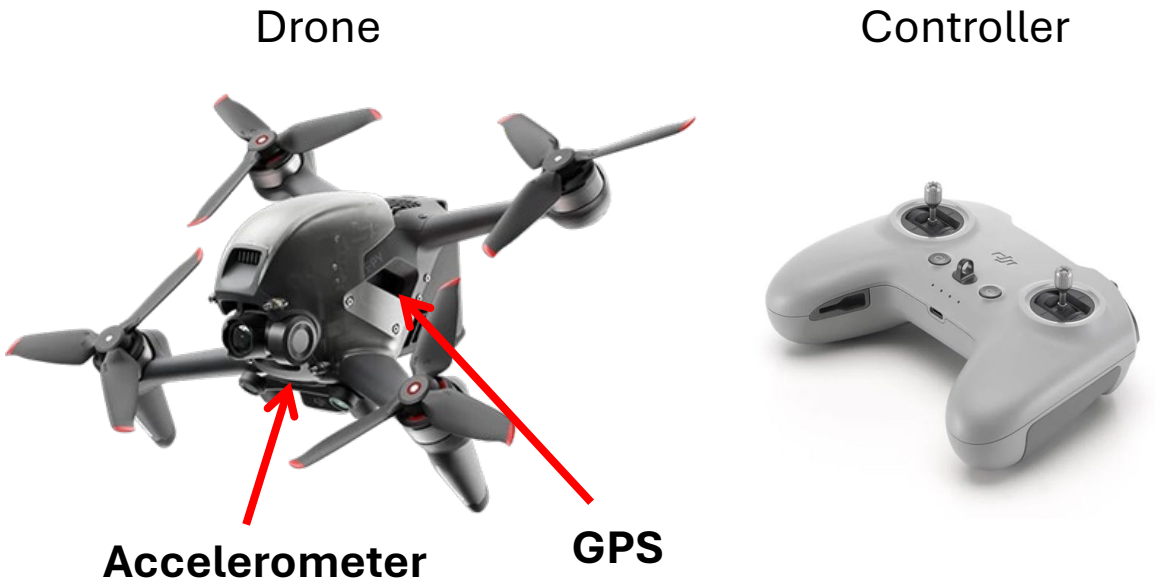
Example: Autonomous Drone

- Can we use **supervised learning** ?
  - Predict the **action** based on the current **state**

State s → Action a

Drone position, velocity, etc.
x

How to move controls sticks (left, right, up, down)
y



- Why Would This Be Hard ?

What is Reinforcement Learning

Example: Autonomous Drone

- Can we use **supervised learning** ?
  - Predict the **action** based on the current **state**

State s → Action a

Drone

Controller

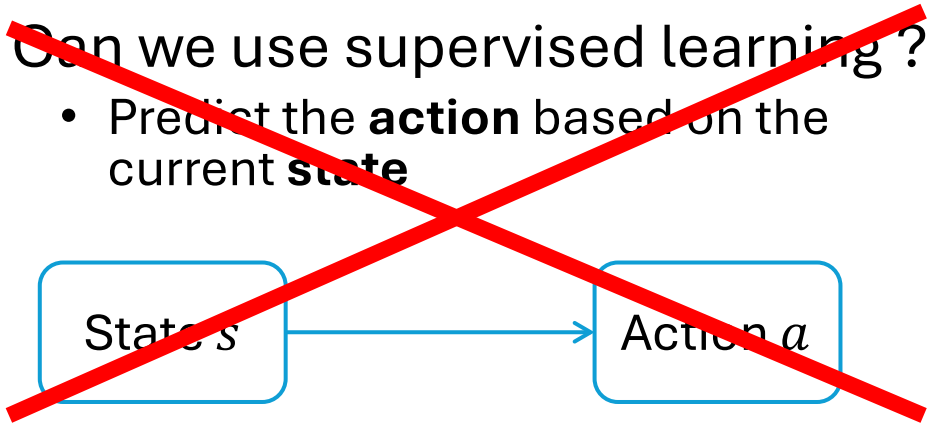
- 1) What is the "correct" label?
  - For each state, what is the best action?
  - The best action depends on future rewards, not just the immediate situation.
  - You often don't know the optimal action in advance.

- 2) Actions affect future data
  - The drone's action changes the next state.
  - Different actions lead to different trajectories.
  - You must evaluate long-term consequences.

What is Reinforcement Learning

Example: Autonomous Drone

- ~~Can we use supervised learning ?~~
  - ~~Predict the **action** based on the current **state**~~



Drone

Controller

- Unlike image classification, there is no ground-truth label readily available.
- This is a sequential decision problem, not independent input-output pairs.

➢ So, we use **Reinforcement Learning (RL)**.

What is Reinforcement Learning

- You define ***what*** good performance is, **not *how*** to achieve it :
  - You reward the drone when it flies well. Example: +1 each second of stable flight.
  - You penalize it (negative reward) when it flies poorly. And you give a large negative reward (-1000) if it crashes.
- The algorithm tries to get more rewards and fewer penalties.
- Over time, it learns to fly well and avoid crashing.

Drone

Controller

- **RL - key idea**
  - Do **not** need to provide the correct action for every state.
  - Instead, provide a **reward** signal.
  - The agent (drone) learns which actions **maximize long-term reward**.

Example: Controlling robots

We really don't know how to tell this robot what the best way is to place its legs in a given situation so that it doesn't fall.

All of this was figured out automatically by the robot, simply by giving it rewards that incentivize making progress toward standing up and not falling.

https://www.youtube.com/watch?v=UpNid_rWDnI

Applications of Reinforcement Learning

- Controlling robots
- Factory optimization
- Financial (stock) trading
- Playing games (including video games)
- etc.

State / Action / Reward

	(terminal state)					(terminal state)
states:	**1**	**2**	**3**	**4**	**5**	**6**
rewards:	**100**	**0**	**0**	**0**	**0**	**40**

actions: LEFT ← RIGHT →

Example:
The robot's goal is to reach a charging station. There are two possible stations: one is fast and efficient (+100 reward), the other works but is less desirable (+40)

State / Action / Reward

	(terminal state)					(terminal state)
states:	**1**	**2**	**3**	**4**	**5**	**6**
rewards:	**100**	**0**	**0**	**0**	**0**	**40**

actions: LEFT ← RIGHT →

(s, a, R(s), s')

(4, LEFT, 0, 3)

Sequence example:

4	**3**	**2**	**1**		
L	**L**	**L**	**-**		
0	**0**	**0**	**100**		

State / Action / Reward

	(terminal state)					(terminal state)
states:	**1**	**2**	**3**	**4**	**5**	**6**
rewards:	**100**	**0**	**0**	**0**	**0**	**40**

actions: LEFT ← RIGHT →

(s, a, R(s), s')

(4, **L**EFT, 0, 3)

Sequence example:

4	**5**	**6**			
R	**R**	**-**			
0	**0**	**40**			

State / Action / Reward

	(terminal state)					(terminal state)
states:	**1**	**2**	**3**	**4**	**5**	**6**
rewards:	**100**	**0**	**0**	**0**	**0**	**40**

actions: LEFT ← RIGHT →

(s, a, R(s), s')

(4, **L**EFT, 0, 3)

Sequence example:

4	**3**	**2**	**3**	**2**	**1**
L	**L**	**R**	**L**	**L**	**-**
0	**0**	**0**	**0**	**0**	**100**

The Return in Reinforcement Learning

- **Question**: How do we compare different sequences of rewards?
  - Is a **small** reward **now** better than a **larger** reward **later**?
  - How do we measure "*better*" over time?

- **Intuition: Immediate vs. Delayed Reward**
  - Imagine:
    - 💵 25 QAR right at your feet
    - 💵 50 QAR half an hour away
  - Which do you choose?
    - 50 QAR is larger
    - But it requires time and effort
    - The 25 QAR is immediate and convenient

➢ Bigger is not always better if it takes too long.

The Return in Reinforcement Learning

- **Key Idea: The Return**
  - In reinforcement learning, we don't just care about the next reward.
  - We care about the **total future reward**, taking into account:
    - How large the rewards are
    - When they are received
  - Rewards received sooner are often more valuable than rewards received much later.

1	2	3	4	5	6
100	0	0	0	0	40

The Return in Reinforcement Learning

- Example: if agent always take left
  - $\text{Return} = 0 + (0.9)\,0 + (0.9)^2\,0 + (0.9)^3\,100 = 72.9$

1	2	3	4	5	6
100	0	0	0	0	40

The Return in Reinforcement Learning

- Example: if agent always take left
  - $\text{Return} = 0 + (0.9)\,0 + (0.9)^2\,0 + (0.9)^3\,100 = 72.9$

- Discount Factor: $\gamma = 0.9$ $\qquad \gamma \in [0, 1]$
- $\mathbf{Return} = \boldsymbol{R_1} + \boldsymbol{\gamma}\,\boldsymbol{R_2} + \boldsymbol{\gamma^2}\,\boldsymbol{R_3} + \boldsymbol{\gamma^3}\,\boldsymbol{R_4} +$ **… (until terminal state)**

1	2	3	4	5	6
100	0	0	0	0	40

The Return in Reinforcement Learning

- Example: if agent always take left
  - Return $= \ 0 \ + \ (0.9)\ 0 \ + \ (0.9)^2\ 0 + (0.9)^3\ 100$ = 72.9

- Discount Factor: $\gamma = 0.9$ $\qquad \gamma \in [0, 1]$
- **Return** $= \boldsymbol{R_1 \ + \gamma\, R_2 \ + \ \gamma^2\, R_3 + \gamma^3\, R_4 +}$ **... (until terminal state)**

- Same example with $\gamma = 0.5$:
  - Return $= \ 0 \ + \ (0.5)\ 0 \ + \ (0.5)^2\ 0 + (0.5)^3\ 100$ = 12.5

1	2	3	4	5	6
100	0	0	0	0	40

The Return in Reinforcement Learning

- With $\gamma = 0.9$:
  - $\text{Return} = 0 + (0.9)\,0 + (0.9)^2\,0 + (0.9)^3\,100$ = **72.9**
- With $\gamma = 0.5$:
  - $\text{Return} = 0 + (0.5)\,0 + (0.5)^2\,0 + (0.5)^3\,100$ = **12.5**

- A **small γ (discount factor)** makes the agent **more impatient**.
  - **Small γ** → future rewards are heavily discounted → focus on immediate rewards
  - **Large γ** → future rewards matter almost as much as immediate ones → more long-term thinking

1	2	3	4	5	6
100	0	0	0	0	40

The Return in Reinforcement Learning

- The return **depends on the actions you take**.
- **Example**: Always taking LEFT
  - Return from state **2** ➔ $0 + (0.5)\,100$ = **50**
  - Return from state **3** ➔ $0 + (0.5)\,0 + (0.5)^2\,100$ = **25**
  - Return from state **4** ➔ $0 + (0.5)\,0 + (0.5)^2\,0 + (0.5)^3\,100$ = **12.5**
  - Return from state **5** ➔ $0 + (0.5)\,0 + (0.5)^2\,0 + (0.5)^3\,0 + (0.5)^4\,100$ = **6.25**

1	2	3	4	5	6	← state
100	50	25	12.5	6.25	40	← return
	←	←	←	←		
100	0	0	0	0	40	← reward

The Return in Reinforcement Learning

- The return **depends on the actions you take**.
- **Example**: Always taking RIGHT
  - Return from state **2** ➔ $0 + (0.5)\,0 + (0.5)^2\,0 + (0.5)^3\,0 + (0.5)^4\,40$ = **2.5**
  - Return from state **3** ➔ $0 + (0.5)\,0 + (0.5)^2\,0 + (0.5)^3\,40$ = **5**
  - Return from state **4** ➔ $0 + (0.5)\,0 + (0.5)^2\,40$ = **10**
  - Return from state **5** ➔ $0 + (0.5)\,40$ = **20**

1	2	3	4	5	6	← state
100	**2.5**	**5**	**10**	**20**	**40**	← **return**
	→	→	→	→		
100	**0**	**0**	**0**	**0**	**40**	← reward

The Return in Reinforcement Learning

- The return **depends on the actions you take**.
- **Example**: Taking RIGHT if we are at state 5, otherwise take LEFT
  - Return from state **2** ➔ $0 + (0.5)\,100$ = **50**
  - Return from state **3** ➔ $0 + (0.5)\,0 + (0.5)^2\,100$ = **25**
  - Return from state **4** ➔ $0 + (0.5)\,0 + (0.5)^2\,0 + (0.5)^3\,100$ = **12.5**
  - Return from state **5** ➔ $0 + (0.5)\,40$ = **20**

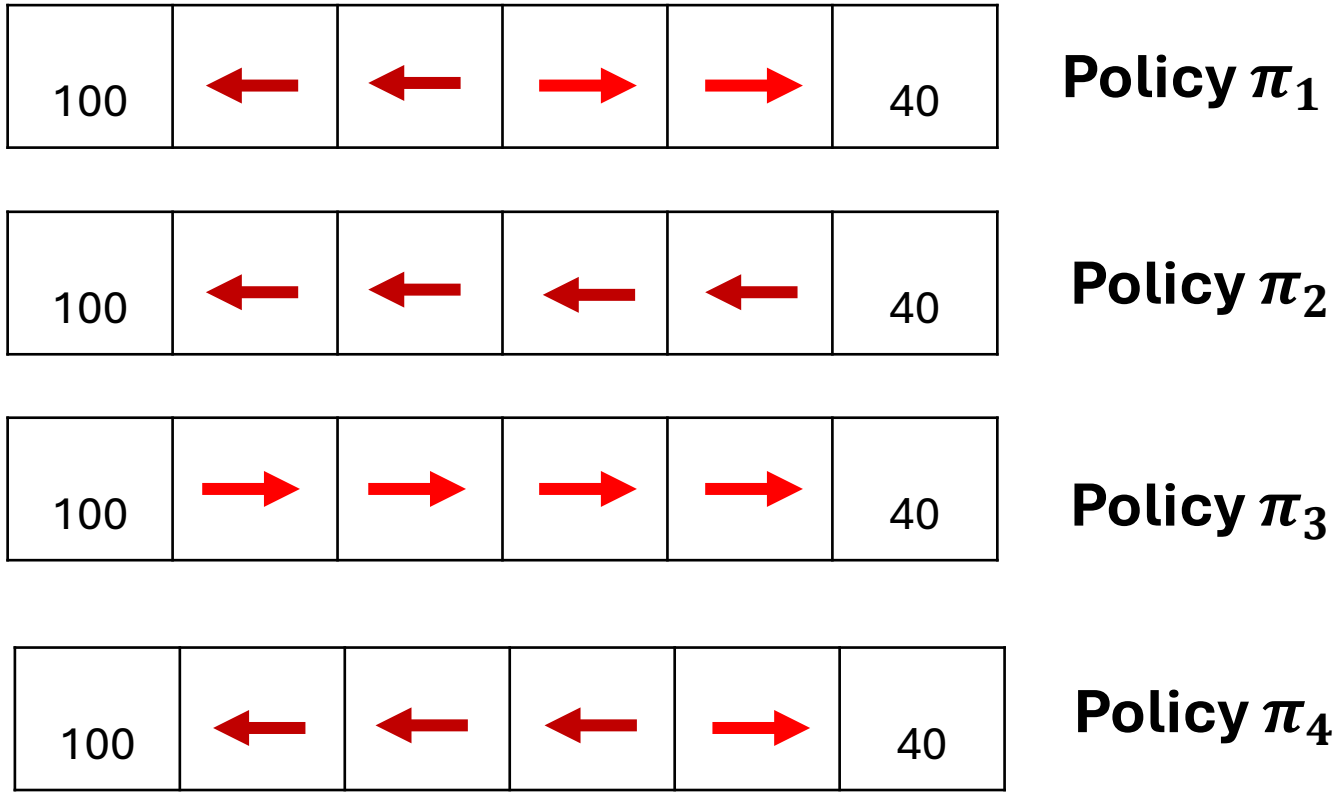
1	2	3	4	5	6	← state
100	50	25	12.5	20	40	← return
	←	←	←	→		
100	0	0	0	0	40	← reward

	1	2	3	4	5	6
Policy A	100 100	50 ← 0	25 ← 0	12.5 ← 0	6.25 ← 0	40 40
Policy B	100 100	2.5 → 0	5 → 0	10 → 0	20 → 0	40 40
Policy C	100 100	50 ← 0	25 ← 0	12.5 ← 0	20 → 0	40 40

Making Decisions: Policies in RL

- A policy is a function $\pi(s) = a$ mapping from states to actions.

- It tells you what action a to take in a given state s.



- **Goal of RL:**
  - Find a policy π that tells you what action ($a = \pi(s)$) to take in every state (s) in order to maximize the return.

Review of Key Concepts

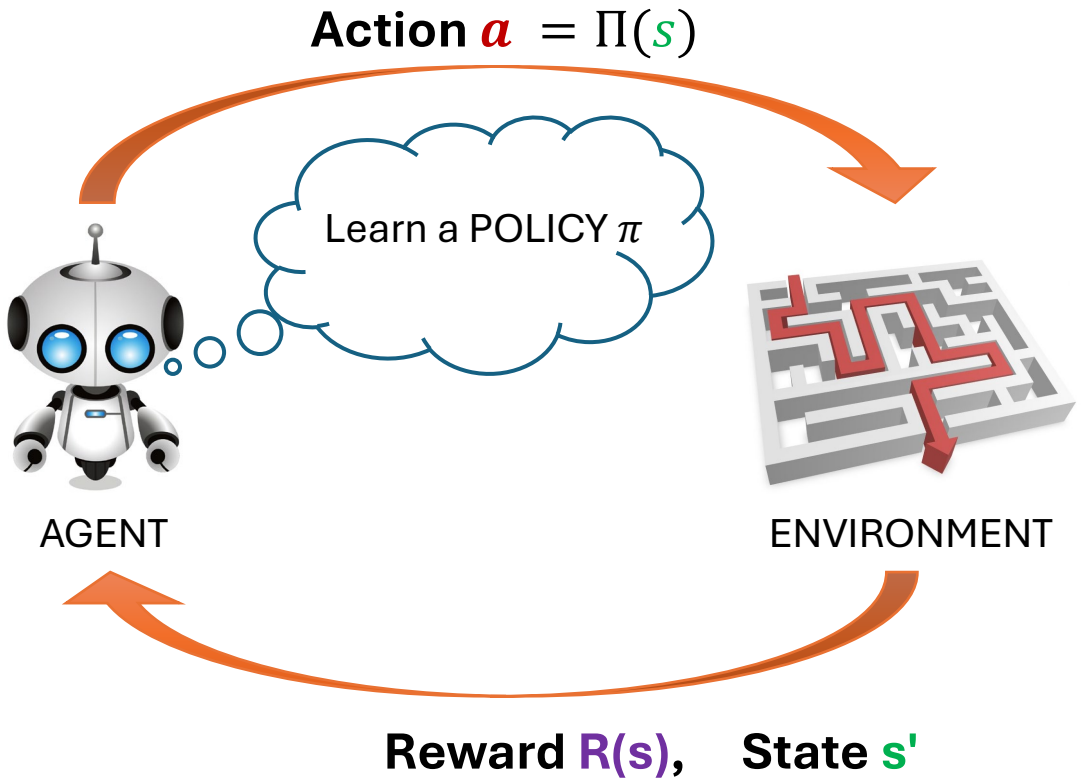
	Our robot	**Drone**	**Chess**
States	6 states	Position of drone	Pieces on board
Actions	left, right ← →	How to move the control stick (up, right, left, down, up-right, ...)	Possible move
Rewards	100, 0, 40	+1, -1000	+1, 0, -1
Discount factor γ	0.5	0.99	0.995
Return	$\boldsymbol{R_1 + \gamma R_2 + \gamma^2 R_3} + \cdots$	$\boldsymbol{R_1 + \gamma R_2 + \gamma^2 R_3} + \cdots$	$\boldsymbol{R_1 + \gamma R_2 + \gamma^2 R_3} + \cdots$
Policy π	100 ← ← ← → 40	Find $\pi(s) = a$	Find $\pi(s) = a$

Review of Key Concepts

- Markov Decision Process (MDP)

(s, a, R(s), s')

Current state — Action — Reward — Next state



Question 1

- You are using reinforcement learning to control a four-legged robot. The *position and direction* of the robot would be its .....

  a) reward
  b) return
  c) state
  d) action

Question 1

- You are using reinforcement learning to control a four-legged robot. The *position and direction* of the robot would be its .....

  a) reward
  b) return
  c) state
  d) action

State = The current situation of the agent (e.g., the robot's position, orientation, joint angles, velocities, etc).
Action = What the agent chooses to do (e.g., move a leg, apply torque).
Reward = Immediate feedback from the environment.
Return = The total accumulated reward over time.

Question 2

- You are controlling a Mars rover. You will be really very happy if it gets to state 1 (significant scientific discovery), slightly happy if it gets to state 2 (small scientific discovery), and unhappy if it gets to state 3 (rover is permanently damaged). To reflect this, choose a reward function so that:

  a) $R(1) > R(2) > R(3)$, where $R(1)$, $R(2)$ and $R(3)$ are negative.
  b) $R(1) > R(2) > R(3)$, where $R(1)$ and $R(2)$ are positive and $R(3)$ is negative.
  c) $R(1) > R(2) > R(3)$, where $R(1)$, $R(2)$ and $R(3)$ are positive.
  d) $R(1) < R(2) < R(3)$, where $R(1)$ and $R(2)$ are negative and $R(3)$ is positive.

Question 2

- You are controlling a Mars rover. You will be really very happy if it gets to state 1 (significant scientific discovery), slightly happy if it gets to state 2 (small scientific discovery), and unhappy if it gets to state 3 (rover is permanently damaged). To reflect this, choose a reward function so that:

  a) R(1) > R(2) > R(3), where R(1), R(2) and R(3) are negative.
  b) R(1) > R(2) > R(3), where R(1) and R(2) are positive and R(3) is negative.
  c) R(1) > R(2) > R(3), where R(1), R(2) and R(3) are positive.
  d) R(1) < R(2) < R(3), where R(1) and R(2) are negative and R(3) is positive.

  - State 1 (significant discovery) → You are really very happy → large positive reward
  - State 2 (small discovery) → You are slightly happy → smaller positive reward
  - State 3 (permanent damage) → You are unhappy → negative reward

Question 3

- You are using RL to fly a drone. Using a discount factor of **0.75**, your drone starts in some state and receives rewards **-100** on the first step, **-100** on the second step, and **1000** on the third and final step (where it has reached a terminal state). What is the return?

  a) -100 - 0.75*100 + 0.75^2*1000
  b) -100 - 0.25*100 + 0.25^2*1000
  c) -0.25*100 - 0.25^2*100 + 0.25^3*1000
  d) -0.75*100 - 0.75^2*100 + 0.75^3*1000

Question 3

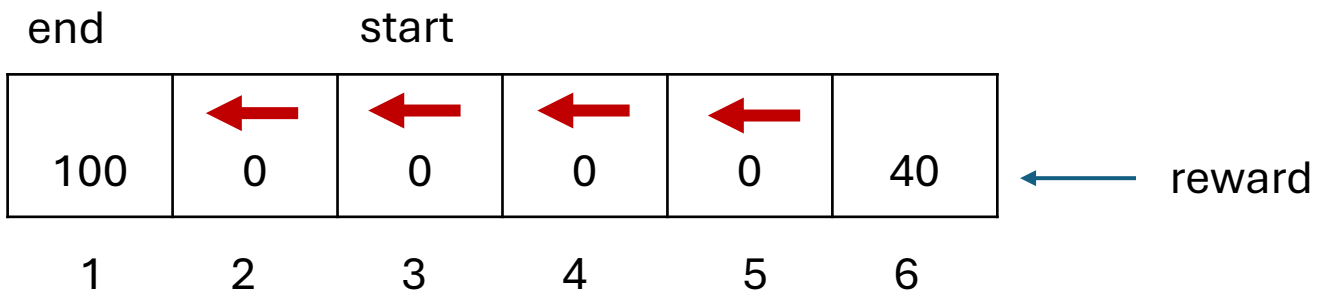
- You are using RL to fly a drone. Using a discount factor of **0.75**, your drone starts in some state and receives rewards **-100** on the first step, **-100** on the second step, and **1000** on the third and final step (where it has reached a terminal state). What is the return?

a) -100 - 0.75*100 + 0.75^2*1000
b) -100 - 0.25*100 + 0.25^2*1000
c) -0.25*100 - 0.25^2*100 + 0.25^3*1000
d) -0.75*100 - 0.75^2*100 + 0.75^3*1000

Question 4

- Given the rewards and actions below, compute the return from state 3 with a discount factor of $\gamma = 0.25$

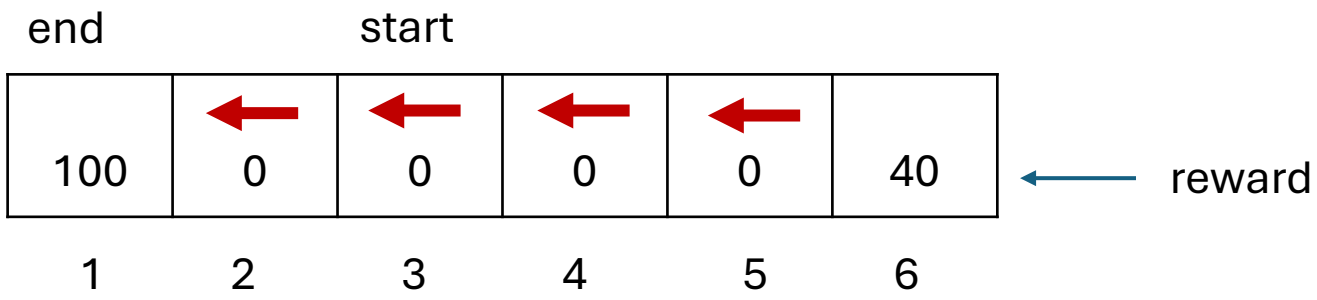
a) 0.39
b) 25
c) 6.25
d) 0



Question 4

- Given the rewards and actions below, compute the return from state 3 with a discount factor of $\gamma = 0.25$

a) 0.39
b) 25
c) 6.25
d) 0



$$0 + 0.25(0) + 0.25^2(100) = 6.25$$

State-Action Value Function (Q-function)

- $Q(s, a)$ = Return if you
  - start from state s
  - take action a (once)
  - then behave optimally after that.

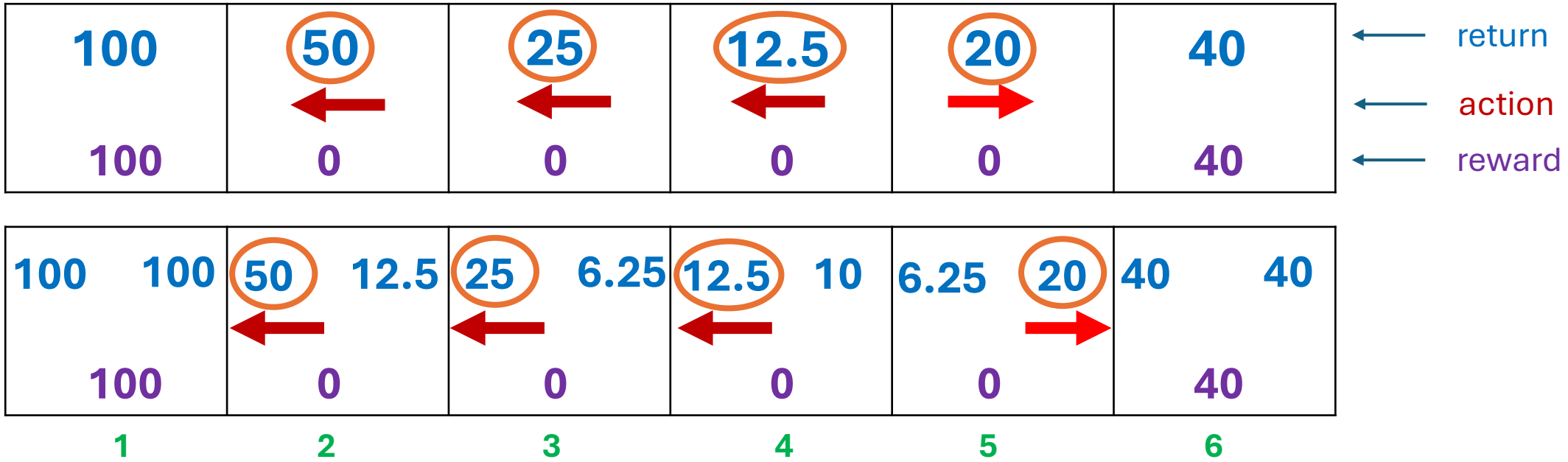
- **Example:**
  - $\boldsymbol{Q}(\mathbf{2}, \leftarrow) = 0 + (0.5) \times 100 = 50$
  - $\boldsymbol{Q}(\mathbf{2}, \rightarrow) = 0 + (0.5) \times 0 + (0.5)^2 \times 0 + (0.5)^3 \times 100 = 12.5$

100	50	25	12.5	20	40	← return
	←	←	←	→		← action
100	0	0	0	0	40	← reward

100 100	50 12.5	25 6.25	12.5 10	6.25 20	40 40
100	0	0	0	0	40
1	2	3	4	5	6

State-Action Value Function (Q-function)

- **Picking actions:**
  - The best possible return from state s is $\max_a \mathrm{Q}(s, a)$
  - The best possible action from state s is the action a that gives $\max_a \mathrm{Q}(s, a)$. It corresponds to the action a given by $a = \pi(s)$



Bellman Equation

- $Q(s,a)$ = Return if you
  - start from state s
  - take action a (once)
  - then behave optimally after that.

- s : current state
- a : current action
- $R(s)$: reward of current state
- s' : state you get to after taking action a
- a' : action that you take in state s'

Bellman equation (simplified) :

- $Q(s,a) = R(s) + \gamma \max_{a'} Q(s',a')$

Example:

- $Q(2,\rightarrow) = R(2) + (0.5) \times \max_{a'} Q(3,a')$

$$= 0 + (0.5) \times 25 = 12.5$$

100 100	50 12.5	25 6.25	12.5 10	6.25 20	40 40
100	0	0	0	0	40
1	2	3	4	5	6

Bellman Equation

- $Q(s,a)$ = Return if you
  - start from state s
  - take action a (once)
  - then behave optimally after that.

- s : current state
- a : current action
- $R(s)$: reward of current state
- s' : state you get to after taking action a
- a' : action that you take in state s'

Bellman equation (simplified) :

- $$Q(s,a) = \underbrace{R(s)}_{\text{Reward you get right away}} + \gamma \underbrace{\max_{a'} Q(s',a')}_{\text{Return from behaving optimally starting from state s'}}$$

Example:

- $$Q(2,\rightarrow) = R(2) + (0.5) \times \max_{a'} Q(3,a')$$
$$= 0 + (0.5) \times 25 = 12.5$$

100 100	50 $\boxed{12.5}$	$\boxed{25}$ 6.25	12.5 10	6.25 20	40 40
100	$\boxed{0}$	0	0	0	40
1	2	3	4	5	6

Random (Stochastic) Environment

1	2	3	4	5	6
100	0	0	0	0	40

Suppose you want to go **LEFT**: ← **0.9** **0.1**

- 90% chance that it correctly goes **left**
- 10% chance that it goes right (e.g., because of motor error, wheel slippage, or imperfect control …)

Random (Stochastic) Environment

1	2	3	4	5	6
100	0	0	0	0	40

Suppose you want to go **RIGHT**: **0.1** → **0.9**

- 90% chance that it correctly goes **right**
- 10% chance that it goes left (e.g., because of motor error, wheel slippage, or imperfect control ...)

Random (Stochastic) Environment

1	2	3	4	5	6
100	0	0	0	0	40

- In this case (stochastic environment), the goal of RL is to learn a policy π such that we maximize the **<u>expected</u> return**.
- **Expected Return:** $\text{Average}(R_1 + \gamma R_2 + \gamma^2 R_3 + \cdots)$
- **Bellman Equation:** $Q(s,a) = R(s) + \gamma E\left[\max_{a'} Q(s',a')\right]$

4 3 or 5

Question 5

- Which of the following accurately describes the state-action value function $Q(s, a)$?

  a) It is the return if you start from state s, take action a (once), then behave optimally after that.
  b) It is the return if you start from state s and repeatedly take action a.
  c) It is the return if you start from state s and behave optimally.
  d) It is the immediate reward if you start from state s and take action a (once).

Question 5

- Which of the following accurately describes the state-action value function $Q(s, a)$?

a) **It is the return if you start from state s, take action a (once), then behave optimally after that.**
b) It is the return if you start from state s and repeatedly take action a.
c) It is the return if you start from state s and behave optimally.
d) It is the immediate reward if you start from state s and take action a (once).

Question 6

- You are controlling a robot that has 3 actions: ← (left), → (right), and STOP. From a given state s, you have computed
  - $Q(s, \leftarrow) = -10$
  - $Q(s, \rightarrow) = -5$
  - $Q(s, \text{STOP}) = 0$

What is the optimal action to take in state s?

a) STOP
b) ← (left)
c) → (right)
d) Impossible to tell

Question 6

- You are controlling a robot that has 3 actions: ← (left), → (right), and STOP. From a given state s, you have computed
  - $Q(s, \leftarrow) = -10$
  - $Q(s, \rightarrow) = -5$
  - $Q(s, \text{STOP}) = 0$

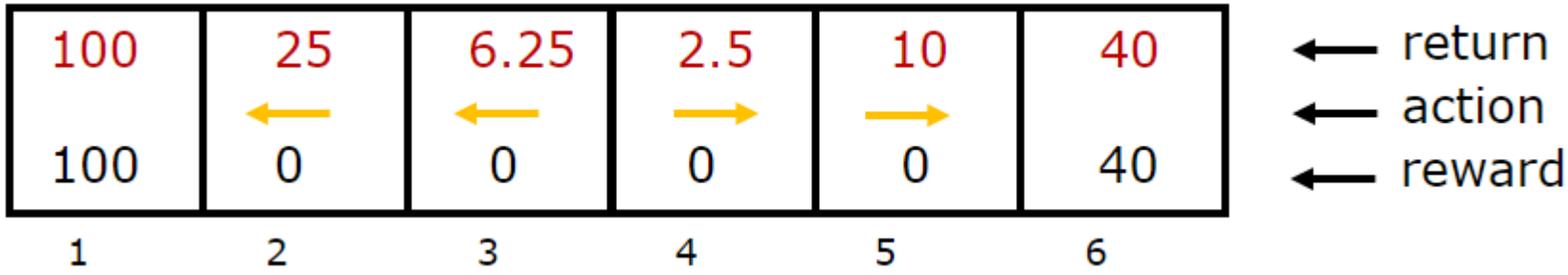
What is the optimal action to take in state s?

a) STOP
b) ← (left)
c) → (right)
d) Impossible to tell

Question 7

- For this problem, $\gamma = 0.25$. The diagram below shows the return and the optimal action from each state. Please compute $Q(5, \leftarrow)$

a) 0.625
b) 0.391
c) 1.25
d) 2.5



Question 7

- For this problem, $\gamma = 0.25$. The diagram below shows the return and the optimal action from each state. Please compute $Q(5, \leftarrow)$

a) **0.625**
b) 0.391
c) 1.25
d) 2.5

100	25	6.25	2.5	10	40
	←	←	→	→	
100	0	0	0	0	40
1	2	3	4	5	6

← return
← action
← reward

Belloman equation : $Q(s,a) = R(s) + \gamma \max_{a'} Q(s', a')$

$$Q(5, \leftarrow) = R(5) + (0.25) \max_{a'} Q(4, a')$$

$$= 0 \quad + (0.25)\, 2.5 \quad = \quad 0.625$$

Basic Q-Learning

- Classic Q-learning assumes finite **discrete** states and action.
- It stores a **Q-Table** with one value per (state, action) pair.
- **Note**: there are variants for continuous states, (e.g., DQN).

	LEFT	**RIGHT**	**etc.**
State 1	$Q(1, \mathrm{L})$	$Q(1, \mathrm{R})$	…
State 2	$Q(2, \mathrm{L})$	$Q(2, \mathrm{R})$	…
State 3	$Q(3, \mathrm{L})$	$Q(3, \mathrm{R})$	…
etc.	…	…	…

Basic Q-Learning

- Classic Q-learning assumes finite **discrete** states and action.
- It stores a **Q-Table** with one value per (state, action) pair.
- **Note**: there are variants for continuous states, (e.g., DQN).

	LEFT	**RIGHT**	**etc.**
State 1	$Q(1, \mathrm{L})$	$Q(1, \mathrm{R})$	…
State 2	$Q(2, \mathrm{L})$	$Q(2, \mathrm{R})$	…
State 3	$Q(3, \mathrm{L})$	$Q(3, \mathrm{R})$	…
etc.	…	…	…

- **Classic Q-Learning:**
  - Agent observes current state s

Basic Q-Learning

- Classic Q-learning assumes finite **discrete** states and action.
- It stores a **Q-Table** with one value per (state, action) pair.
- **Note**: there are variants for continuous states, (e.g., DQN).

	LEFT	**RIGHT**	**etc.**
State 1	$Q(1, \mathrm{L})$	$Q(1, \mathrm{R})$	…
State 2	$Q(2, \mathrm{L})$	$Q(2, \mathrm{R})$	…
State 3	$Q(3, \mathrm{L})$	$Q(3, \mathrm{R})$	…
etc.	…	…	…

- **Classic Q-Learning:**
  - Agent observes current state s
  - Chooses an action :
    - With small probability ϵ, it explores a random action a.
    - With probability $1 - \epsilon$, it exploits $a = \arg\max_{\mathrm{a}} Q\,(s, a)$

Basic Q-Learning

- Classic Q-learning assumes finite **discrete** states and action.
- It stores a **Q-Table** with one value per (state, action) pair.
- **Note**: there are variants for continuous states, (e.g., DQN).

	LEFT	**RIGHT**	**etc.**
State 1	$Q(1, \mathrm{L})$	$Q(1, \mathrm{R})$	…
State 2	$Q(2, \mathrm{L})$	$Q(2, \mathrm{R})$	…
State 3	$Q(3, \mathrm{L})$	$Q(3, \mathrm{R})$	…
etc.	…	…	…

- **Classic Q-Learning:**
  - Agent observes current state s
  - Chooses an action :
    - With small probability ϵ, it explores a random action a.
    - With probability $1 - \epsilon$, it exploits $a = \arg\max_{\mathrm{a}} Q\,(s, a)$
  - Receives a reward $R(s)$ and moves to a new state s'.

Basic Q-Learning

- Classic Q-learning assumes finite **discrete** states and action.
- It stores a **Q-Table** with one value per (state, action) pair.
- **Note**: there are variants for continuous states, (e.g., DQN).

	LEFT	**RIGHT**	**etc.**
State 1	$Q(1, \mathrm{L})$	$Q(1, \mathrm{R})$	...
State 2	$Q(2, \mathrm{L})$	$Q(2, \mathrm{R})$	...
State 3	$Q(3, \mathrm{L})$	$Q(3, \mathrm{R})$	...
etc.	...	...	...

- **Classic Q-Learning:**
  - Agent observes current state s
  - Chooses an action :
    - With small probability ϵ, it explores a random action a.
    - With probability $1 - \epsilon$, it exploits $a = \arg\max_{\mathrm{a}} Q\,(s, a)$
  - Receives a reward $R(s)$ and moves to a new state s'.
  - Updates the Q-value using the rule:
    - new_value $\leftarrow$ **old_value** $+\ \alpha$ [**target_value** $-$ **old_value**]

Basic Q-Learning

	LEFT	RIGHT	etc.
State 1	$Q(1, \mathrm{L})$	$Q(1, \mathrm{R})$	...
State 2	$Q(2, \mathrm{L})$	$Q(2, \mathrm{R})$	...
State 3	$Q(3, \mathrm{L})$	$Q(3, \mathrm{R})$	...
etc.	...	...	...

- Classic Q-learning assumes finite **discrete** states and action.
- It stores a **Q-Table** with one value per (state, action) pair.
- **Note**: there are variants for continuous states, (e.g., DQN).

Reminder - Bellman equation: $Q(s,a) = \boldsymbol{R(s)} + \boldsymbol{\gamma \max_{a'} Q(s', a')}$

- **Classic Q-Learning:**
  - Agent observes current state s
  - Chooses an action :
    - With small probability ϵ, it explores a random action a.
    - With probability $1 - \epsilon$, it exploits $a = \arg\max_{\mathrm{a}} Q\,(s, a)$
  - Receives a reward $R(s)$ and moves to a new state s'.
  - Updates the Q-value using the rule:
    - new_value $\leftarrow$ **old_value** $+\ \alpha$ [**target_value** $-$ **old_value**]

Basic Q-Learning

- Classic Q-learning assumes finite **discrete** states and action.
- It stores a **Q-Table** with one value per (state, action) pair.
- **Note**: there are variants for continuous states, (e.g., DQN).

	LEFT	RIGHT	etc.
State 1	$Q(1,\mathrm{L})$	$Q(1,\mathrm{R})$	...
State 2	$Q(2,\mathrm{L})$	$Q(2,\mathrm{R})$	...
State 3	$Q(3,\mathrm{L})$	$Q(3,\mathrm{R})$	...
etc.	...	...	...

Reminder - Bellman equation: $Q(s,a) = \boldsymbol{R(s)} + \boldsymbol{\gamma \max_{a'} Q(s',a')}$

$\boldsymbol{\max_{a'} Q(s',a')}$ come from your current estimates at state $\boldsymbol{s'}$ (from your current Q-Table).

- **Classic Q-Learning:**
  - Agent observes current state s
  - Chooses an action :
    - With small probability ϵ, it explores a random action a.
    - With probability $1-\epsilon$, it exploits $a = \arg\max_{a} Q\,(s,a)$
  - Receives a reward $R(s)$ and moves to a new state s'.
  - Updates the Q-value using the rule:
    - new_value $\leftarrow$ **old_value** $+\ \alpha$ [**target_value** $-$ **old_value**]

Basic Q-Learning

- Classic Q-learning assumes finite **discrete** states and action.
- It stores a **Q-Table** with one value per (state, action) pair.
- **Note**: there are variants for continuous states, (e.g., DQN).

	LEFT	RIGHT	etc.
State 1	$Q(1,\mathrm{L})$	$Q(1,\mathrm{R})$	...
State 2	$Q(2,\mathrm{L})$	$Q(2,\mathrm{R})$	...
State 3	$Q(3,\mathrm{L})$	$Q(3,\mathrm{R})$	...
etc.	...	...	...

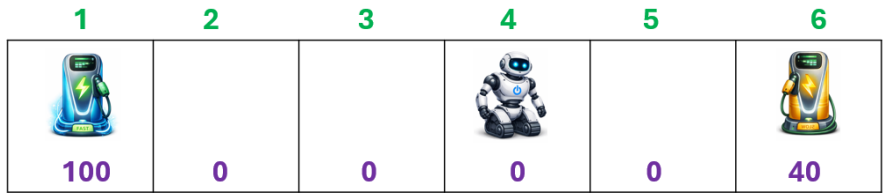
Reminder - Bellman equation: $Q(s,a) = \boldsymbol{R(s)} + \boldsymbol{\gamma \max_{a'} Q(s',a')}$

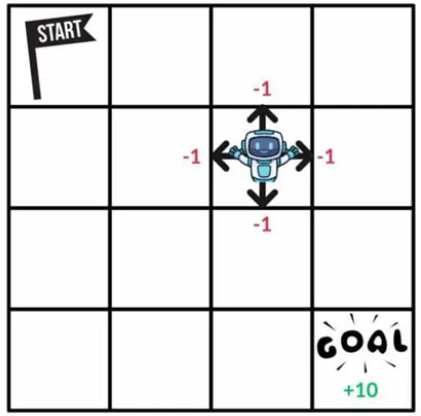
$\boldsymbol{\max_{a'} Q(s',a')}$ come from your current estimates at state $\boldsymbol{s'}$ (from your current Q-Table).

- **Classic Q-Learning:**
  - Agent observes current state s
  - Chooses an action :
    - With small probability ϵ, it explores a random action a.
    - With probability $1-\epsilon$, it exploits $a = \arg\max_{a} Q\,(s,a)$
  - Receives a reward $R(s)$ and moves to a new state s'.
  - Updates the Q-value using the rule:
    - new_value $\leftarrow$ **old_value** $+\ \alpha$ [**target_value** $-$ **old_value**], so in other words:
    - $Q(s,a) \leftarrow \boldsymbol{Q(s,a)} + \alpha\left[\boldsymbol{R(s)} + \boldsymbol{\gamma \max_{a'} Q(s',a')} - \boldsymbol{Q(s,a)}\right]$

Continuous State Space

- So far, we considered a **discrete state** space





- But often we have **continuous state** spaces (various features such as real-valued measurements):

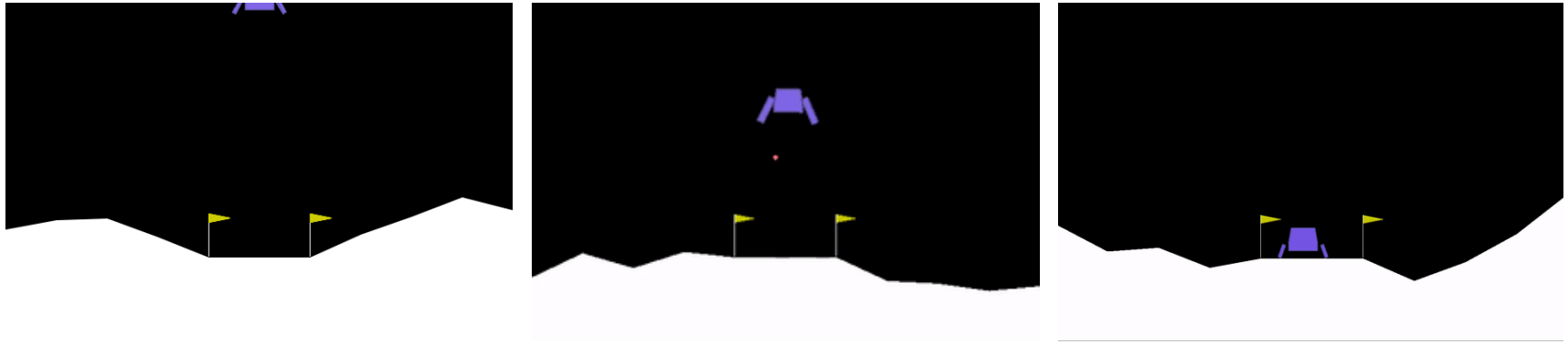
  - **Robot position and velocity**: state = (x position, y position, speed)
  - **Self-driving car**: state = (car position, orientation angle, velocity, distance to lane center)
  - **Robotic arm**: state = (joint angles, joint angular velocities)
  - **Drone control**: state = (x, y, z position, velocities, roll, pitch, yaw)
  - **Finance / trading agent**: state = (price, volatility, portfolio value)
  - etc.

Continuous State Space

- In your lab you will use environments with continuous states. For example, the **Lunar Lander** environment:

Continuous State Space

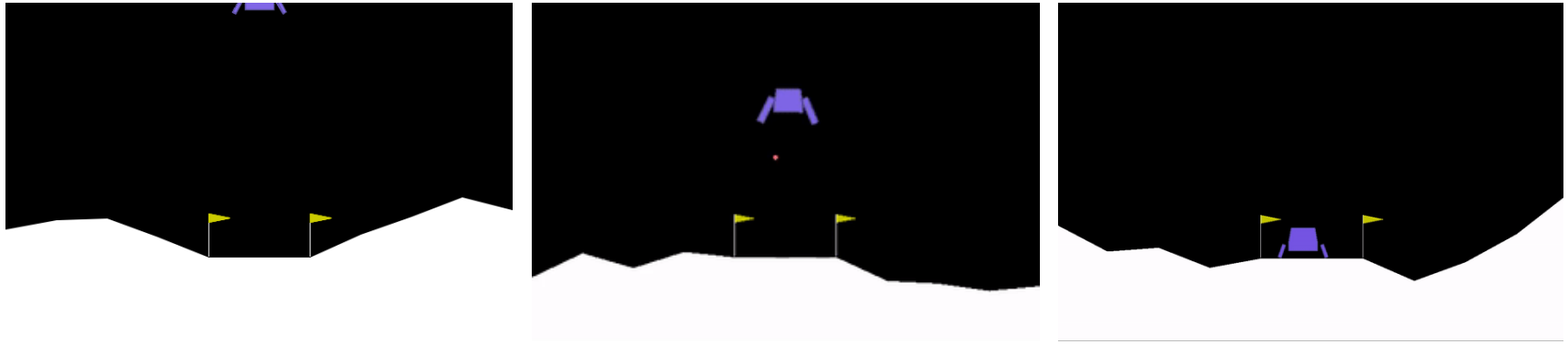
- In your lab you will use environments with continuous states. For example, the **Lunar Lander** environment:



See: https://gymnasium.farama.org/environments/box2d/lunar_lander/

Continuous State Space

- In your lab you will use environments with continuous states. For example, the **Lunar Lander** environment:



See: https://gymnasium.farama.org/environments/box2d/lunar_lander/

State - 8-dimensional vector:

- Coordinates (x, y)
- Linear velocities in (x, y)
- Angle, and angular velocity
- Two Booleans indicating if each leg is in contact with the ground or not.

Actions (discrete) :

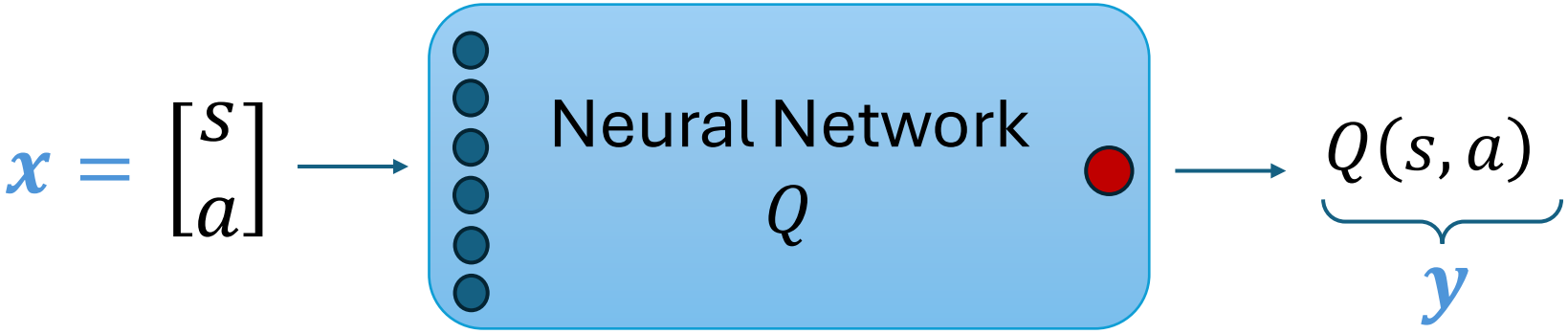
- **0**: do nothing
- **1**: fire left orientation engine
- **2**: fire main engine
- **3**: fire right orientation engine

- **Rewards:**
  - Land: 100-140
  - Reward for moving toward/away from pad
  - Crash: -100
  - Soft landing: +10
  - Fire main engine: -0.3
  - Fire side engine: -0.03 …

Deep Reinforcement Learning (Deep Q Net)

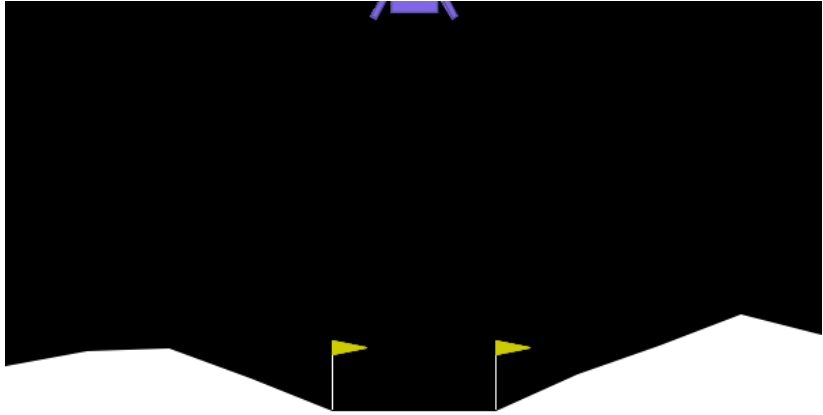
Bellman equation: $\underbrace{Q(s,a)}_{\boldsymbol{x}} = \underbrace{R(s) + \gamma \max_{a'} Q(s',a')}_{\boldsymbol{y}}$

- **Learning the state-value function**
  - Train a neural network to take as input $\boldsymbol{x} = (s,a)$, and predict as output $\boldsymbol{y} = R(s) + \gamma \max_{a'} Q(s',a')$



- In a state s, use neural network to compute $Q(s,a_i)$, $i = 0, 1, 2, \ldots$
- Pick the action a_i that maximizes $Q(s,a_i)$

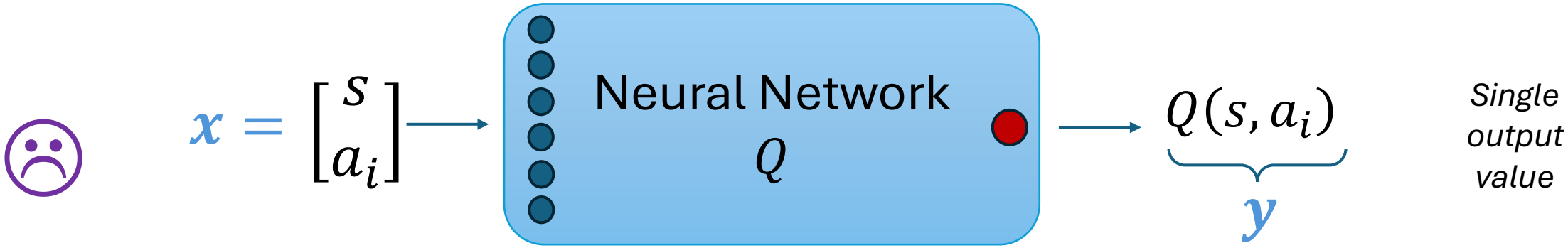
Deep Reinforcement Learning



Simplified DQN Algorithm:

- Initialize a neural network randomly as guess of $Q(s, a)$
- Repeat:
  - Take actions in the environment. Get $(s, a, R(s), s')$.
  - Store 10000 most recent $(s, a, R(s), s')$ tuples. ← ***Reply Buffer***
  - Train neural network:
    - Create training s et of 10000 examples using $\boldsymbol{x} = (\boldsymbol{s}, \boldsymbol{a})$ and $\boldsymbol{y} = \boldsymbol{R}(\boldsymbol{s}) + \boldsymbol{\gamma} \max_{\boldsymbol{a}'} \boldsymbol{Q}(\boldsymbol{s}', \boldsymbol{a}')$
    - Train the Q_{new} such that $Q_\theta(s, a) \approx y$
  - Set $Q = Q_{new}$

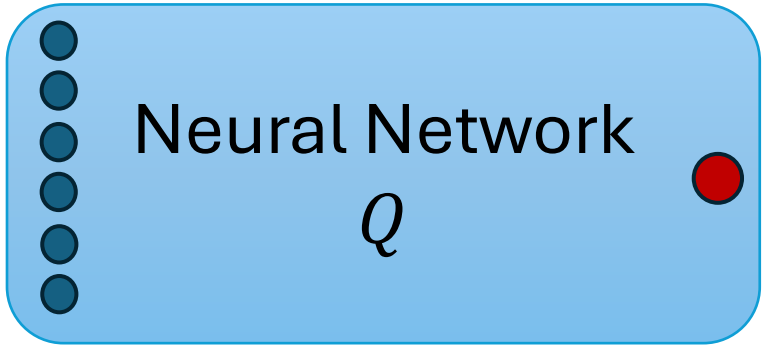
Deep Q Network – Improved version (1)



You need several forwards at inference: try input s with action a_i

Deep Q Network – Improved version (1)

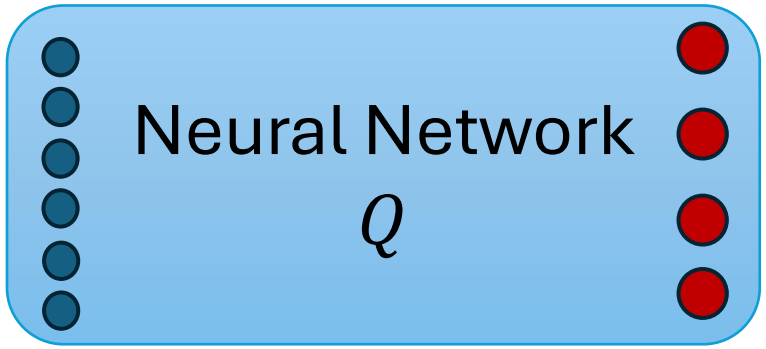
$$\boldsymbol{x} = \begin{bmatrix} s \\ a_i \end{bmatrix}$$



$$\underbrace{Q(s, a_i)}_{\boldsymbol{y}}$$

Single output value

$$\boldsymbol{x} = s$$



$$\begin{pmatrix} Q(s, a_0) \\ Q(s, a_1) \\ Q(s, a_2) \\ Q(s, a_3) \end{pmatrix}$$

$$\boldsymbol{y}$$

Output vector

Deep Q Network – Improved version (2)

Simplified DQN Algorithm:

- Initialize a neural network randomly as guess of $Q(s, a)$
- Repeat:
  - **Take actions in the environment**. Get $(s, a, R(s), s')$.
  - Store 10000 most recent $(s, a, R(s), s')$ tuples.

  - Train neural network:
    - Create training s et of 10000 examples using $\boldsymbol{x} = (\boldsymbol{s}, \boldsymbol{a})$ and $\boldsymbol{y} = \boldsymbol{R}(\boldsymbol{s}) + \boldsymbol{\gamma} \max_{\boldsymbol{a}'} \boldsymbol{Q}(\boldsymbol{s}', \boldsymbol{a}')$
    - Train the Q_{new} such that $Q_\theta(s, a) \approx y$
  - Set $Q = Q_{new}$

How to choose actions while still learning ?

Deep Q Network – Improved version (2)

How to choose actions while still learning ?

- Use an ϵ-greedy policy.

- In some state s
  - **Option 1:**
    - Pick the action a that maximizes $Q(s, a)$
      - This is full exploitation. Not always good because you never explore some states and actions.
  - **Option 2:**
    - With probability ϵ (e.g. 0.05) pick an action a randomly ➔ **Exploration**
    - With probability $1 - \epsilon$ (e.g. 0.95), pick the action a that maximizes $Q(s, a)$ ➔ **Exploitation**

- Usually, we start training using a high ϵ, and we gradually decrease it over time (explore more at the beginning and less at the end).

Deep Q Network – Improved version (3)

Simplified DQN Algorithm:

- Initialize a neural network randomly as guess of $Q(s, a)$
- Repeat:
  - Take actions in the environment. Get $(s, a, R(s), s')$.
  - Store 10000 most recent $(s, a, R(s), s')$ tuples.

  - Train neural network:
    - Create training s et of 10000 examples using $\boldsymbol{x} = (\boldsymbol{s}, \boldsymbol{a})$ and $\boldsymbol{y} = \boldsymbol{R}(\boldsymbol{s}) + \boldsymbol{\gamma} \max_{\boldsymbol{a}'} \boldsymbol{Q}(\boldsymbol{s}', \boldsymbol{a}')$
    - Train the Q_{new} such that $Q_\theta(s, a) \approx y$
  - **Set $\boldsymbol{Q} = \boldsymbol{Q_{new}}$**

Hard assignment ➔ Less stable learning
Can cause sudden shifts in Q-value targets, leading to training instability.

Deep Q Network – Improved version (3)

- **Hard update**: Set $Q \leftarrow Q_{new}$

  Parameters θ $\quad$ θ_{new}

  - **Less stable learning**: Hard update (copying parameters) can cause sudden shifts in Q-value targets, leading to training instability compared to smoother or soft updates.

- **Soft update**:
  - $\theta \leftarrow \tau\, \theta_{new} + (1 - \tau)\, \theta$

    $\tau \in [0, 1]$: the soft update coefficient

  - Example: $\theta \leftarrow 0.2\, \theta_{new} + 0.8\, \theta$
  - If $\tau = 1$, then it's the hard update version.

Limitations of Reinforcement Learning

- **Simulation-to-reality gap**: Policies trained in simulation often perform poorly when deployed in real-world environments.
- **High sample complexity**: RL typically requires a very large number of interactions with the environment to learn effective policies.
- **Limited real-world deployment**: Compared to supervised and unsupervised learning, RL currently has fewer widely adopted applications.
- **Training instability**: Learning can be unstable and sensitive to hyperparameters and reward design.
- But … exciting topic with a lot of potential for future applications.

Question 8

- The Lunar Lander is a continuous state Markov Decision Process (MDP) because:

  a) The state-action value $Q(s, a)$ function outputs continuous valued numbers
  b) The state has multiple numbers rather than only a single number (such as position in the x-direction)
  c) The state contains numbers such as position and velocity that are continuous valued.
  d) The reward contains numbers that are continuous valued

Question 8

- The Lunar Lander is a continuous state Markov Decision Process (MDP) because:

  a) The state-action value $Q(s, a)$ function outputs continuous valued numbers
  b) The state has multiple numbers rather than only a single number (such as position in the x-direction)
  c) The state contains numbers such as position and velocity that are continuous valued.
  d) The reward contains numbers that are continuous valued

Question 9

- In the learning algorithm described for DQN, we repeatedly create an artificial training set to which we apply supervised learning, where the input is $x = (s, a)$ and the target, constructed using Bellman's equations, is $y = \ldots\ldots\ldots$?

  a) $\mathrm{y} = \max_{\mathrm{a}'} \mathrm{Q}(\mathrm{s}', \mathrm{a}')$, where s' is the state reached after taking action a in state s
  b) $\mathrm{y} = \mathrm{R}(\mathrm{s}) + \gamma \max_{\mathrm{a}'} \mathrm{Q}(\mathrm{s}', \mathrm{a}')$, where s' is the state reached after taking action a in state s
  c) $\mathrm{y} = \mathrm{R}(\mathrm{s}')$, where s' is the state reached after taking action a in state s
  d) $\mathrm{y} = \mathrm{R}(\mathrm{s})$

Question 9

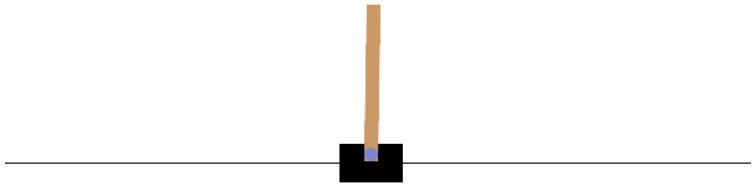
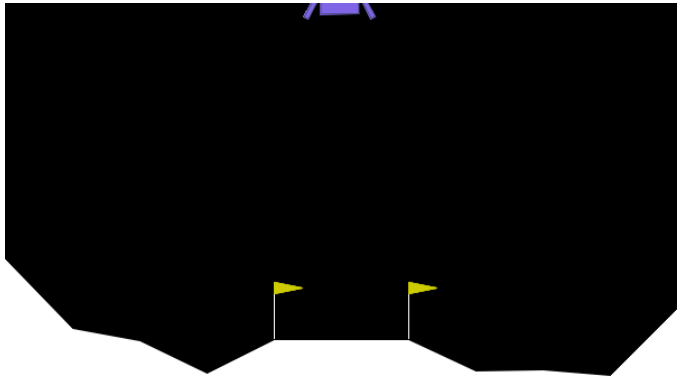
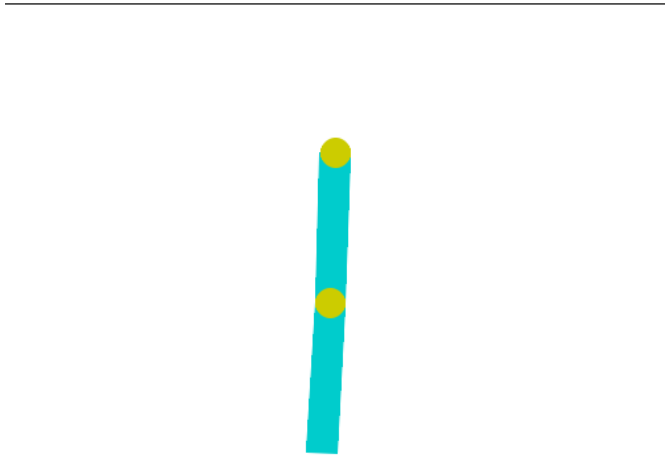
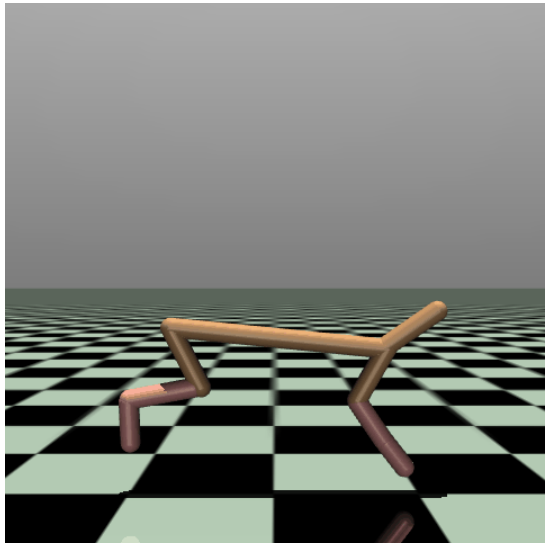
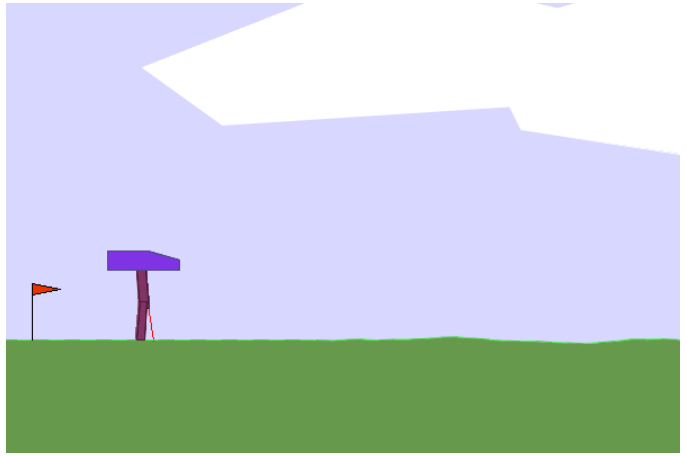
- In the learning algorithm described for DQN, we repeatedly create an artificial training set to which we apply supervised learning, where the input is $x = (s, a)$ and the target, constructed using Bellman's equations, is $y = \ldots\ldots\ldots$?

a) $y = \max_{a'} Q(s', a')$, where s' is the state reached after taking action a in state s

b) $y = R(s) + \gamma \max_{a'} Q(s', a')$, where s' is the state reached after taking action a in state s

c) $y = R(s')$, where s' is the state reached after taking action a in state s

d) $y = R(s)$

Lab

- Use DQN and various other RL methods, in various environments …
- Use Google Colab for this Lab (not locally).